KLASTROWANIE SEMANTYCZNE · WARSTWA ZNACZENIA

KILKA REDAKCJI. JEDNA HISTORIA.

> **Embedding** to wektor, czyli lista liczb o stałej długości, którą model językowy oblicza dla danego fragmentu tekstu. Teksty o podobnym znaczeniu zwykle otrzymują wektory położone blisko siebie, więc system może zmierzyć ich podobieństwo bez subiektywnej oceny czytelnika.

Kiedy na przykład prezydent rezygnuje z urzędu, kilka redakcji opisuje to wydarzenie, każda po swojemu. Teksty mogą nie mieć wspólnych sformułowań, a czasem są napisane w różnych językach. Program, który grupuje artykuły na podstawie wspólnych słów, nie rozpozna nawet prostej parafrazy.

System przedstawia każdy artykuł jako punkt w przestrzeni znaczeń. Nie mierzy zwykłej odległości między punktami. Zamiast tego porównuje kierunki prowadzących do nich wektorów za pomocą cosinusa kąta. Wynik pozwala oszacować, czy teksty są ze sobą powiązane.

Grupa tematyczna, do której należy artykuł, ma w bazie własny wiersz i istnieje przez wiele dni. Nowe doniesienie nie jest porównywane z każdym innym artykułem, tylko ze środkami istniejących grup. Wyszukuje je baza danych PostgreSQL z rozszerzeniem **pgvector**.

768
LICZB NA ARTYKUŁ
Tyle współrzędnych opisuje położenie każdego tekstu

0,70
PRÓG DOŁĄCZENIA DO GRUPY
Podobieństwo cosinusowe mieści się w przedziale od −1,00 do 1,00. Artykuł dołącza do grupy, gdy jego podobieństwo do środka grupy wynosi co najmniej 0,70; w pokazanej próbce doniesienie o tym samym wydarzeniu uzyskuje względem środka wynik 0,9933, a doniesienie o innym wydarzeniu 0,4010

3
TEKSTY NA WYWOŁANIE
Serwer może mieć 4 GB pamięci i nie korzystać z pamięci wymiany (swapu), dlatego teksty trafiają do modelu małymi porcjami

brak
TERMINU WAŻNOŚCI
Sam upływ czasu nie wymaga ponownego obliczenia zapisanego wektora

ELEMENTARZ

ZNACZENIE ZAPISANE JAKO WSPÓŁRZĘDNE

Trzy punkty poniżej pokazują, jak z tekstu powstaje embedding, jak porównuje się embeddingi i dlaczego raz obliczony wynik można zachować.

01 TEKST JAKO 768 WSPÓŁRZĘDNYCH

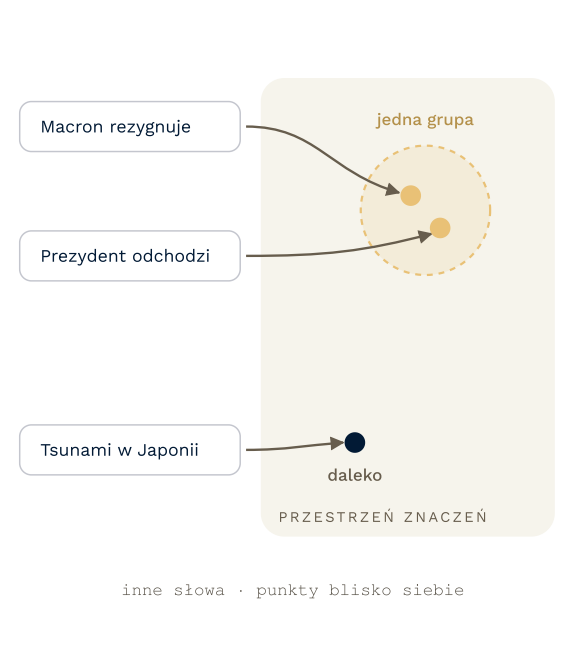
Model zamienia zdanie w listę 768 liczb. Wynik nie jest streszczeniem ani zestawem słów kluczowych. Te liczby są współrzędnymi, które wyznaczają położenie tekstu w przestrzeni znaczeń. Każdy tekst trafia do tej samej przestrzeni.

02 BLISKOŚĆ WSKAZUJE NA POKREWIEŃSTWO

Teksty o tym samym wydarzeniu często trafiają prawie w to samo miejsce, mimo że używają innych słów. To „prawie” da się zmierzyć: służy do tego **cosinus kąta** między wektorami. Cały rachunek pokazuje karta „Jak liczy się ten wynik” pod spodem. Dzięki embeddingom nie trzeba ręcznie pisać reguł dotyczących synonimów i szyku zdania.

03 RAZ OBLICZONY WEKTOR MOŻNA WYKORZYSTAĆ PONOWNIE

Jeśli wersja modelu i sposób obliczania embeddingów pozostają bez zmian, ten sam tekst daje powtarzalny wynik. Warto więc zapisać wektor i odczytywać go, gdy tekst pojawia się ponownie, bez kolejnego uruchamiania modelu.

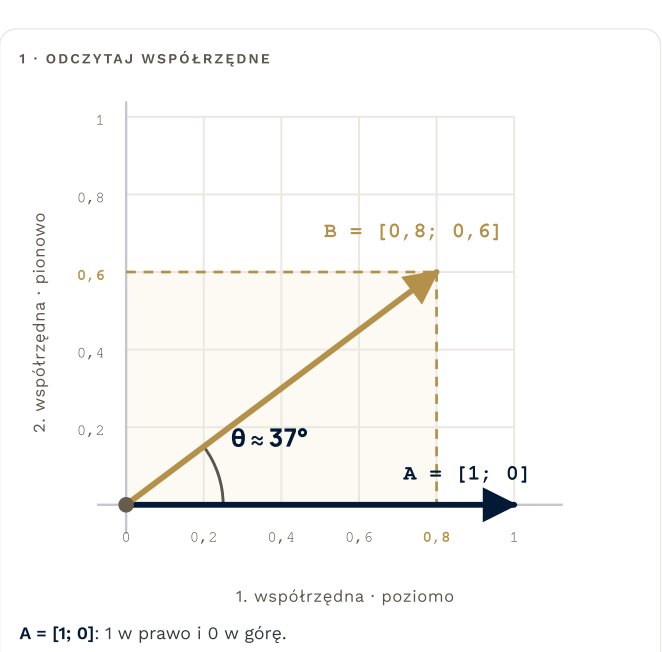


inne słowa · punkty blisko siebie

JAK LICZY SIĘ TEN WYNIK

Miara z punktu 02 to **cosinus kąta** między wektorami, czyli to samo podobieństwo cosinusowe, na którego skali leży próg 0,70. Wektor to lista liczb; tutaj bierzemy dwie współrzędne zamiast 768, żeby cały rachunek zmieścił się na jednym rysunku. Wektory A i B dobraliśmy do przykładu; nie pochodzą z modelu. Obie strzałki zaczynają się w punkcie [0; 0]: pierwsza współrzędna mówi, jak daleko przesunąć koniec strzałki w poziomie, a druga w pionie. Samego kąta nie trzeba znać — jego cosinus liczy się wprost ze współrzędnych obu strzałek.

1 · ODCZYTAJ WSPÓŁRZĘDNE



A = [1; 0]: 1 w prawo i 0 w górę.
B = [0,8; 0,6]: 0,8 w prawo i 0,6 w górę.

2 · POLICZ PODOBIEŃSTWO KIERUNKÓW

$$\cos(\theta) = \frac{A \cdot B}{|A| \times |B|}$$

$A \cdot B$ to iloczyn skalarny: pomnóż pary liczb i dodaj wyniki.

$|A|$ i $|B|$ to długości strzałek. Obie muszą być większe od zera, bo nie można dzielić przez zero.

1. Policz licznik. Pomnóż liczby na tych samych pozycjach i dodaj: $(1 \times 0{,}8) + (0 \times 0{,}6) = 0{,}8$
2. Oblicz długości wektorów z twierdzenia Pitagorasa: podnieś współrzędne do kwadratu, dodaj wyniki i wyciągnij pierwiastek: $|A| = \sqrt{(1^2 + 0^2)} = 1$; $|B| = \sqrt{(0{,}8^2 + 0{,}6^2)} = \sqrt{(0{,}64 + 0{,}36)} = 1$
3. Podziel licznik przez iloczyn długości. Dzielenie usuwa wpływ rozmiaru strzałek, więc zostaje porównanie ich kierunków: $0{,}8 \div (1 \times 1) = 0{,}80$

0,80 Wynik 0,80 opisuje podobieństwo kierunków. Nie oznacza 80% pewności, że teksty opisują to samo wydarzenie. W sekcji „Grupy tematyczne” ta miara posłuży do dopasowania artykułu do grupy.

−1 · przeciwne 0 · prostopadłe 1 · ten sam kierunek

Na rysunku widać kąt około 37°, ponieważ $\arccos(0{,}80) \approx 37°$. System nie potrzebuje samego kąta. Do porównania tekstów używa bezpośrednio wyniku 0,80.

> **OD STRONY TECHNICZNEJ**
>
> W używanym tu modelu wektor ma 768 współrzędnych. Aby obliczyć iloczyn skalarny, program mnoży 768 par liczb i dodaje wyniki. Ollama przez `/api/embed` zwraca wektory znormalizowane do długości 1 (normalizacja L2), z możliwymi drobnymi odchyleniami wynikającymi z precyzji obliczeń. Dla wektorów o długości 1 mianownik wynosi 1, więc cosinus jest równy iloczynowi skalarnemu. Pełny wzór działa również dla wektorów o innych, niezerowych długościach.

PUNKT ODNIESIENIA

DLACZEGO NIE WYSTARCZY LICZENIE SŁÓW

Zanim upowszechniły się embeddingi, do porównywania tekstów często stosowano metodę **TF-IDF**. Najpierw liczy ona wystąpienia słów, a następnie przypisuje każdemu z nich wagę zależną od częstości w danym tekście i rzadkości w całym zbiorze. Dwa teksty uznaje za podobne, gdy występują w nich te same słowa o wysokich wagach.

JAK TF-IDF PORÓWNUJE TEKSTY

O wadze każdego słowa decydują dwie wielkości: częstość występowania w tym tekście oraz rzadkość w całym zbiorze. Słowo obecne wszędzie, takie jak „oraz” albo „jest”, nie odróżnia jednego tekstu od drugiego i otrzymuje wagę bliską zeru. Słowo rzadkie, takie jak „rezygnacja”, otrzymuje dużą wagę.

Tekst zamienia się w listę takich wag, a podobieństwo dwóch tekstów jest miarą tego, jak bardzo obie listy się pokrywają. Nie potrzeba tu modelu językowego ani uczenia maszynowego. Całość sprowadza się do obliczeń opartych na częstości słów, które można zamknąć w kilkudziesięciu wierszach kodu i uruchomić bez żadnych przygotowań.

KIEDY TF-IDF NIE ROZPOZNAJE PODOBIEŃSTWA

Problem pojawia się, gdy dwa teksty mówią to samo innymi słowami. „Prezydent ustępuje” i „Macron rezygnuje” nie mają ani jednego wspólnego, istotnego słowa, więc ich podobieństwo wynosi dokładnie zero.

Żeby metoda rozpoznała podobieństwo tych zdań, trzeba dopisać listę synonimów, reguły sprowadzania słów do form podstawowych i wykaz nazwisk. Ktoś musi przygotować je ręcznie, a potem powtórzyć tę pracę dla każdego języka z osobna.

NAJTRUDNIEJSZY PRZYPADEK: KILKA JĘZYKÓW NARAZ

To samo wydarzenie może być jednocześnie opisywane po angielsku, francusku i polsku. Jeśli artykuły w dwóch językach nie mają wspólnych słów, metoda oparta na ich liczeniu nada im podobieństwo równe zeru, nawet gdy opisują to samo wydarzenie. Bez dodatkowych reguł trzeba by najpierw przetłumaczyć wszystko na jeden język. Wymagałoby to kolejnego modelu, a jego błędy wpływałyby na wyniki.

W języku polskim dochodzi jeszcze odmiana: „rezygnacja”, „rezygnacji” i „rezygnacją” to dla licznika trzy różne słowa. Model wielojęzyczny może porównywać teksty bez osobnych reguł dla każdego języka. Ponieważ uczył się wielu języków naraz, umieszcza napisane w nich teksty we wspólnej przestrzeni. Polskie zdanie może więc znaleźć się blisko swojego angielskiego odpowiednika.

PYTANIE	TF-IDF	EMBEDDINGI
Co porównuje?	wspólne słowa	sens całego tekstu
Skąd wie, co jest ważne?	z tego, jak rzadkie są słowa w bieżącym zbiorze tekstów	z modelu, który wcześniej przeczytał ogromną liczbę tekstów
Rozpozna inne sformułowanie tej samej rzeczy?	nie, dopóki ktoś nie dopisze słownika synonimów	zwykle tak, ponieważ model został wytrenowany do porównywania znaczenia
Poradzi sobie z dwoma językami?	nie bez dodatkowej obsługi, na przykład tłumaczenia	tak, jeśli model jest wielojęzyczny
A z nazwiskiem, symbolem giełdowym, numerem ustawy?	lepiej: dokładne trafienie rzadkiego słowa waży najwięcej	gorzej: podobne nazwy potrafią wylądować blisko siebie
Czy wynik jest zawsze taki sam?	nie, zmienia się razem ze zbiorem, w którym liczona jest rzadkość słów	wynik jest powtarzalny, jeśli model i sposób obliczania embeddingów pozostają bez zmian
Ile kosztuje?	niewiele; obliczenia odbywają się lokalnie, bez dodatkowych narzędzi	model zajmuje 1,3 GB, a obliczenie trwa kilka sekund; zapisany wynik można wykorzystywać ponownie
Da się sprawdzić, dlaczego tak wyszło?	tak, widać, które słowa zadecydowały	nie; to 768 liczb, z których żadna z osobna nic nie znaczy

Żadna z tych metod nie jest bezwzględnie lepsza. Wyszukiwarki zwykle łączą oba podejścia: liczenie słów dobrze rozpoznaje numer ustawy, a embeddingi oddają sens pytania. Tutaj trzeba ustalić, czy artykuły opisują to samo wydarzenie, więc liczy się przede wszystkim ich sens.

DLACZEGO ZOSTAŁA TYLKO JEDNA METODA

Początkowo sposób grupowania zależał od ustawienia: środowisko programisty bez modelu korzystało z TF-IDF, a środowisko produkcyjne z embeddingów. W rezultacie ta sama porcja artykułów układała się w tych środowiskach w inne grupy, a ten sam próg podobieństwa stosowano do dwóch różnych rozkładów wyników. Gdy na podstawie grup artykułów powstają sygnały mogące wspierać decyzje inwestycyjne, „u mnie wyszło inaczej” przestaje być drobiazgiem.

Liczenie słów miało tylko jedną przewagę: nie wymagało modelu. Zniknęła, gdy raz obliczono embeddingi dla danych testowych i zapisano je w pliku dołączonym do kodu. Przed każdym scenariuszem testowym gotowe wektory są wczytywane do pamięci podręcznej. Testy korzystają więc z tych samych embeddingów i tej samej metody grupowania co środowisko produkcyjne, ale nie muszą wywoływać modelu.

MODEL I INFRASTRUKTURA

SKĄD BIORĄ SIĘ TE LICZBY I GDZIE DZIAŁA MODEL?

Model musiał obsługiwać wiele języków, zmieścić się na jednym serwerze z bazą danych i aplikacją oraz działać we własnej infrastrukturze, bez wysyłania treści na zewnątrz.

NOMIC-EMBED-TEXT-V2-MOE

To otwarty model, który zamienia tekst na 768 liczb. Nie prowadzi rozmowy, nie streszcza tekstów ani nie odpowiada na pytania.

Skrót **MoE** w nazwie to *mixture of experts*: model zawiera kilka wyspecjalizowanych podsieci, ale dla każdego fragmentu tekstu uruchamia tylko część z nich. Ma więc możliwości większego modelu, choć zużywa zasoby na poziomie mniejszego.

W tym zadaniu liczy się przede wszystkim wielojęzyczność: francuska depesza i jej angielskie omówienie mogą trafić blisko siebie w przestrzeni, bez tłumaczenia i bez ręcznie napisanych reguł.

DANE TECHNICZNE

PARAMETRY MODELU

WYJŚCIE	768 liczb · da się skrócić kosztem jakości
JĘZYKI	wielojęzyczny · około stu języków
ILE CZYTA NARAZ	~512 tokenów (~2 akapity) · dalsza część jest obcinana
WAGI	otwarte · pobierane raz, trzymane lokalnie
W PAMIĘCI	~1,3 GB · tylko na czas liczenia porcji
GDZIE DZIAŁA	Ollama na tym samym serwerze · bez połączeń z siecią zewnętrzną

Model czyta około 512 tokenów, więc dalsza część tekstu jest obcinana. W tym systemie przyjęto, że temat artykułu powinien być widoczny w nagłówku i pierwszym akapicie. Jeśli istotna informacja pojawi się później, nie wpłynie na wektor.

? CZYM JEST OLLAMA

Ollama to program do uruchamiania modeli językowych na własnym komputerze albo serwerze. Ollamę instaluje się raz, a model pobiera się jednym poleceniem. Model działa wtedy pod lokalnym adresem, podobnie jak baza danych, do której aplikacja wysyła zapytanie i z której otrzymuje wynik.

Ollama otrzymuje listę tekstów i zwraca listę wektorów. Aplikacja wywołuje ją przez HTTP tylko wtedy, gdy trzeba obliczyć nowy wynik. Żaden tekst nie wychodzi przy tym poza serwer, a za lokalne wywołanie nie płaci się osobno.

ZŁ CZEGO WYMAGA URUCHOMIENIE

Model tej wielkości zajmuje w pamięci ~1,3 GB, wyłącznie na czas liczenia porcji tekstów. Mieści się obok bazy danych i samej aplikacji na zwykłym VPS-ie za kilkadziesiąt złotych miesięcznie. Nie wymaga karty graficznej ani konta u dostawcy chmury.

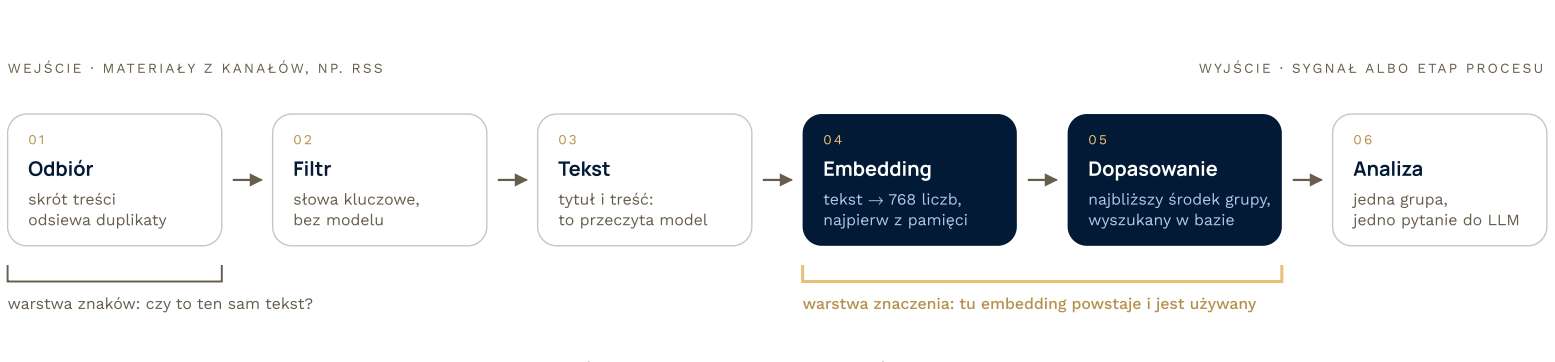
Serwer może mieć 4 GB pamięci i nie korzystać z pamięci wymiany (swapu). W takiej konfiguracji model jest usuwany z pamięci zaraz po policzeniu porcji, a teksty trafiają do niego po trzy. Ponowne załadowanie trwa kilka sekund i jest potrzebne tylko wtedy, gdy porcja zawiera nową treść.

> **OD STRONY TECHNICZNEJ**
>
> `POST /api/embed`, po 3 teksty w paczce, `keep_alive: 0` przy ostatniej paczce (zwolnienie pamięci). Adres i nazwa modelu pochodzą z konfiguracji (`OLLAMA_EMBED_MODEL`).

OBIEG

JAKĄ ROLĘ W TYM PROCESIE ODGRYWA EMBEDDING

Artykuł przechodzi sześć kroków od kanału informacyjnego do sygnału. Tylko dwa wymagają embeddingu. Pozostałe ograniczają liczbę wywołań modelu i pomagają uzyskać wiarygodny wynik.


WEJŚCIE · MATERIAŁY Z KANAŁÓW, NP. RSS
WYJŚCIE · SYGNAŁ ALBO ETAP PROCESU
01 Odbiór skrót treści odsiewa duplikaty
02 Filtr słowa kluczowe, bez modelu
03 Tekst tytuł i treść: to przeczyta model
04 Embedding tekst → 768 liczb, najpierw z pamięci
05 Dopasowanie najbliższy środek grupy, wyszukany w bazie
06 Analiza jedna grupa, jedno pytanie do LLM
warstwa znaków: czy to ten sam tekst?
warstwa znaczenia: tu embedding powstaje i jest używany

Kroki od 01 do 03 nie analizują znaczenia: pierwszy porównuje znaki, drugi szuka słów z listy, a trzeci składa tekst dla modelu. Analiza znaczenia zaczyna się w kroku 04. Jej wynik zostaje zapisany jako wiersz liczb i wykorzystany w kroku 05, gdzie baza danych wskazuje grupę o najbliższym środku. Krok 06 otrzymuje grupę z nowymi artykułami, nie wektor.

≡ ODBIÓR (KROK 01): CZY TEN TEKST JUŻ TU BYŁ?

Ta sama depesza może przyjść z dwóch kanałów albo ponownie po przerwaniu połączenia. Program wykrywa duplikat za pomocą **skrótu treści**, czyli krótkiego ciągu znaków obliczonego z całego tekstu. Ten sam tekst zawsze daje ten sam skrót. Zmiana tekstu w praktyce daje inny skrót. Jeśli taki skrót jest już w bazie, program nie zapisuje artykułu ponownie.

Rozstrzygnięcie jest zerojedynkowe i zupełnie nie uwzględnia znaczenia: przesunięcie choćby jednego przecinka daje inny skrót i drugi wiersz w bazie.

≈ EMBEDDING I GRUPOWANIE (KROKI 04 I 05): CZY TO TA SAMA HISTORIA?

Kilka redakcji pisze o jednym wydarzeniu, ale każda używa innych słów. Dlatego teksty mają różne skróty. Dopiero embedding pozwala zmierzyć, jak blisko siebie leżą te teksty, i połączyć je w jedną grupę, mimo że nie mają ani jednego wspólnego zdania.

Każde z tych pytań wymaga innego narzędzia. Najpierw program porównuje skróty treści. To szybka operacja, która nie wymaga modelu. Dopiero teksty, które przeszły ten etap, dostają wektor i są porównywane ze środkami grup, które baza już zna. Wtorkowe doniesienie może dołączyć do grupy założonej w poniedziałek, która zachowuje swój identyfikator.

01 PRZYJMIJ, ALE TYLKO RAZ

Artykuły przychodzą z kanałów informacyjnych, czasem więcej niż raz. Duplikat może pojawić się, gdy system automatycznie ponawia wysyłkę po przekroczeniu czasu oczekiwania albo gdy ta sama depesza trafia do dwóch kanałów. W obu przypadkach skrót treści jest taki sam, więc program odrzuca drugi egzemplarz.

> **OD STRONY TECHNICZNEJ**
>
> `WorldEvent` · o tożsamości wydarzenia rozstrzyga identyfikator obliczany z nagłówka i treści za pomocą szybkiej, niekryptograficznej funkcji skrótu (`xxh128(title + content)`). To klucz **deduplikacji**, celowo inny niż ten, którym posługuje się pamięć podręczna embeddingów.

WEJŚCIE · materiały z RSS **WYJŚCIE** · utrwalone wydarzenia

02 ODRZUĆ TO, CO NA PEWNO JEST NIEISTOTNE

Następnie prosty filtr sprawdza słowa wymagane i dyskwalifikujące oraz, opcjonalnie, czy źródło znajduje się na liście zaufanych. Nie wymaga modelu i ogranicza liczbę tekstów, dla których trzeba go uruchomić.

Jeśli nikt nie skonfigurował słów kluczowych, nie przechodzi nic. Przepuszczenie wszystkich tekstów mogłoby ukryć błąd konfiguracji aż do otrzymania rachunku.

Listę trzeba jednak najpierw przygotować. Duży model językowy może pomóc podczas konfiguracji, ale nie bierze udziału w przetwarzaniu. Operator opisuje, czego szuka, a model przygotowuje warianty, formy pokrewne i odpowiedniki w językach materiałów źródłowych. Operator sprawdza i zatwierdza ostateczną listę.

> **OD STRONY TECHNICZNEJ**
>
> `WorldEventFilter → WorldEventFilterSpec`: filtr oparty na wyrażeniach regularnych, działający wyłącznie na CPU. Przy pustej konfiguracji nie przepuszcza żadnego tekstu, zamiast domyślnie przepuszczać wszystkie. Podpowiadanie słów to osobna ścieżka, wywoływana z ekranu konfiguracji: `KeywordSuggesterInterface::suggest(opis, języki)`, adapter `AnthropicKeywordSuggester` z pamięcią podręczną i limitem liczby zapytań.

WEJŚCIE · migawka wydarzenia **WYJŚCIE** · array: obiekty WorldEventDescriptor

03 USTAL, CO PRZECZYTA MODEL

Dla każdego artykułu kod łączy nagłówek z treścią. Model otrzyma właśnie ten tekst; od jego brzmienia zależą zarówno wektor, jak i klucz w pamięci podręcznej.

> **OD STRONY TECHNICZNEJ**
>
> `FilteredWorldEvent::getTextForClustering()` → `title . ' ' . content`.

WYJŚCIE · array: teksty

04 ZAMIEŃ TEKST NA LICZBY

Obliczenie wektora zużywa więcej zasobów niż pozostałe kroki, dlatego system najpierw sprawdza, czy tekst został już wcześniej przeliczony. Model otrzymuje wyłącznie teksty, dla których nie ma zapisanego wyniku.

WEJŚCIE · array: teksty **WYJŚCIE** · array: wektory po 768 floatów

> **OD STRONY TECHNICZNEJ**
>
> W PHP wynik ma typ `array`: zawiera wektory, a każdy z nich jest tablicą 768 liczb typu `float`. PHPDoc opisuje tę strukturę dokładniej jako `list<list<float>>`.

05 DOŁĄCZ DO GRUPY ALBO ZAŁÓŻ NOWĄ

Program szuka grupy najbardziej podobnej do nowego artykułu. Jeśli podobieństwo wynosi co najmniej 0,70, dołącza artykuł do tej grupy; w przeciwnym razie zakłada nową. Sekcja „Grupy tematyczne” pokazuje, jak baza wyszukuje dopasowanie i jak zmienia się środek grupy.

> **OD STRONY TECHNICZNEJ**
>
> Opis dotyczy dopasowania do trwałych grup tematycznych, czyli klastrów zapisanych w bazie. Podczas obserwacji procesów wieloetapowych `ProcessStepper` korzysta z tego samego podziału: filtruje wydarzenia według słów kluczowych danego etapu, a pozostałe grupuje według identyfikatora nadanego w tym kroku. Nie tworzy osobnych klastrów.

WEJŚCIE · array: wektory po 768 floatów **WYJŚCIE** · wydarzenie → identyfikator grupy

06 JEDNA GRUPA, JEDNA ANALIZA

Na ostatnim etapie program wykonuje jedną analizę dla całej grupy doniesień o tym samym wydarzeniu, zamiast rozpatrywać każdy artykuł osobno. Wynikiem jest sygnał transakcyjny albo informacja, że dłuższy proces przeszedł do kolejnego etapu. Kiedy analiza grupy da sygnał, kolejne doniesienia o tym samym dołączają do grupy, ale nie wywołują nowej analizy, dopóki ten sygnał obowiązuje.

> **OD STRONY TECHNICZNEJ**
>
> Klaster wraca do analizy tylko wtedy, gdy przybył mu nowy artykuł. Przerwy po sygnale pilnuje `SignalCooldownPolicy`. Przy obserwacji procesu etap nie pyta drugi raz o klaster, który już ocenił, chyba że doszedł do niego artykuł pasujący do tego etapu. Kroki 05 i 06 mają osobne przełączniki: można dalej grupować artykuły, a wstrzymać płatną analizę.

WYJŚCIE · Signal **WYJŚCIE** · obserwacja etapu

ZAPAMIĘTYWANIE

JAK WYNIKI MODELU TRAFIAJĄ DO BAZY

Pamięć podręczna zapisuje każdy obliczony wektor, aby model nie liczył go ponownie. Jedna klasa zarządza pamięcią i przekazuje modelowi teksty, dla których brakuje wyniku.

NAJPIERW PYTANIE, POTEM LICZENIE

Program oblicza skrót każdego tekstu, podobnie jak przy wykrywaniu duplikatów. Tym razem podstawą jest dokładnie ta treść, którą otrzyma model. Następnie program wysyła skróty całej porcji do bazy w jednym zapytaniu i sprawdza, które już w niej są.

Do modelu trafiają tylko pozostałe teksty. Program zapisuje każdy wynik, zanim go użyje. Gdy ten sam tekst pojawi się ponownie, odczyta gotowy wektor z bazy.

WZORZEC: DEKORATOR

Klasa, która odczytuje dane z bazy, nie oblicza embeddingów. Klasa obliczająca embeddingi nie zna bazy. Obie udostępniają ten sam interfejs, więc pierwsza może opakować drugą. Tak działa wzorzec dekoratora.

Pozostała część programu zleca jedynie zamianę tekstów na wektory i nie musi wiedzieć, czy wynik pochodzi z bazy, czy z modelu. Pamięć podręczną można dzięki temu usunąć, wymienić lub uzupełnić o kolejną warstwę bez zmiany kodu, który z niej korzysta.

PRZYKŁAD DEKORATORA W KODZIE

W tej skróconej wersji pominięto logowanie, sprawdzanie długości odpowiedzi i obsługę powtórzonych tekstów w jednej porcji.

```php
final readonly class CachedTextVectorizer implements TextVectorizerInterface
{
    public function __construct(
        private TextVectorizerInterface $inner,    // model (Ollama)
        private EmbeddingStoreInterface $store,    // tabela embedding_cache
        private string $modelId,
        private bool $ollamaEnabled,
    ) {}

    public function vectorize(array $texts): array
    {
        $hashOf  = array_map(self::keyFor(...), $texts);
        $vectors = $this->store->getMany($hashOf, $this->modelId);   // 1 zapytanie

        $misses = $this->collectMisses($texts, $hashOf, $vectors);

        if ([] !== $misses) {
            if (!$this->ollamaEnabled) {
                // brak wektora to błąd, nigdy cichy powrót do innej metody
                throw new \RuntimeException('Embedding cache miss …');
            }

            $fresh = $this->inner->vectorize($misses);       // jedyne wywołanie modelu
            $this->store->putMany($fresh, $this->modelId);  // zapis przed użyciem
            $vectors += $fresh;
        }

        return array_map(static fn (string $h) => $vectors[$h], $hashOf);
    }

    public static function keyFor(string $text): string
    {
        return hash('sha256', trim($text));   // klucz z tekstu dla modelu
    }
}
```

Klucz w bazie to para `(skrót tekstu, nazwa modelu)`, a wiersz zawiera wektor zapisany jako `jsonb`. Wynik nie ma terminu ważności. Sam upływ czasu nie wymaga ponownego obliczenia wektora.

TRZY MOŻLIWE SCENARIUSZE

TEKST JUŻ ZNANY
dowolny tryb
Gdy ten sam tekst pojawia się ponownie, program odczytuje liczby z bazy bez uruchamiania modelu.

NOWY TEKST, MODEL DOSTĘPNY
wywołania modelu włączone
Model wykonuje obliczenia lokalnie, po trzy teksty naraz. Wynik od razu trafia do bazy, więc przy ponownym przetwarzaniu tego tekstu nie trzeba już uruchamiać modelu.

NOWY TEKST, MODEL WYŁĄCZONY
wywołania modelu wyłączone
Program zatrzymuje się, wskazuje tekst, dla którego brakuje wyniku, oraz podaje polecenie pozwalające go przygotować. Nie używa w zamian słabszej metody.

> **OD STRONY TECHNICZNEJ**
>
> Tabela `embedding_cache`, klucz złożony `(sha256(trim(text)), model_id)`, wektor jako `jsonb`, bez TTL. Przy zapisie używane jest `ON CONFLICT DO NOTHING`, więc równoległe procesy robocze nie wchodzą w konflikt. Nazwa modelu jest częścią klucza: po jej zmianie program nie odczytuje już wcześniejszych wierszy dla tego samego tekstu. Wektor jest liczony od nowa. Wektory poprzedniego modelu zostają w tabeli, ale nie są już odczytywane, więc poprawność nie zależy od tego, czy ktoś pamięta o wyczyszczeniu pamięci podręcznej. Jeśli zmieni się wersja modelu, ale jego nazwa pozostanie ta sama, program nadal będzie korzystał z wektorów zapisanych dla poprzedniej wersji. Ustawienie: `OLLAMA_ENABLED`. Inicjalizacja pamięci podręcznej: `bin/console app:embed:warm --days=1`.

GRUPY TEMATYCZNE

JAK ARTYKUŁ TRAFIA DO GRUPY TEMATYCZNEJ

Grupa tematyczna to zbiór artykułów, które system uznał za doniesienia o tym samym wydarzeniu. Gdy przychodzi kolejny tekst, program musi zdecydować: dołączyć go do znanej grupy czy założyć osobną? Korzysta przy tym z embeddingów, czyli liczbowych reprezentacji tekstów.

CZYM JEST „ŚRODEK GRUPY”

Każdy artykuł ma wektor złożony z 768 liczb. Żeby uzyskać jeden wektor dla całej grupy, program liczy średnią osobno na każdej pozycji: z pierwszych liczb wszystkich artykułów, potem z drugich i tak dalej. Wynikiem jest kolejna lista 768 liczb. To właśnie środek grupy.

Środek służy jako wspólny punkt odniesienia dla grupy. Program porównuje z nim wektor nowego tekstu. Przy jednym artykule środek jest równy jego wektorowi; przy dwóch jest ich średnią.

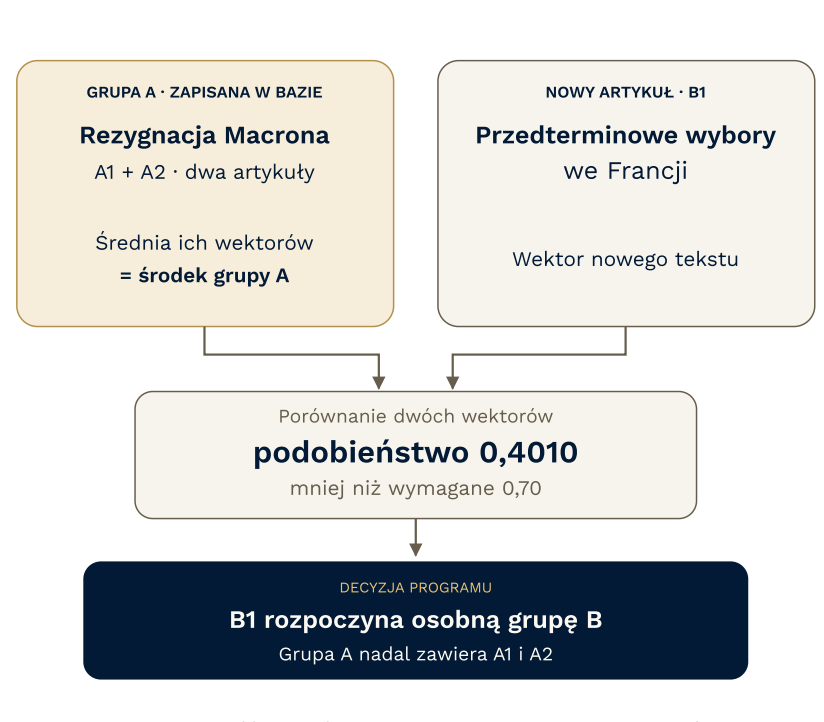
W CZYM POMAGA PGVECTOR

Model zamienia tekst na liczby. pgvector, rozszerzenie bazy PostgreSQL, pozwala zapisać te liczby i porównywać wektory bezpośrednio w bazie. Tutaj baza porównuje wektor nowego artykułu ze środkami zapisanych grup i zwraca najlepsze dopasowania.

Program sprawdza następnie, czy podobieństwo do najlepiej dopasowanej grupy jest wystarczająco wysokie. Samo znalezienie najbardziej podobnej grupy nie wystarcza, żeby przypisać do niej artykuł.

CZY WIADOMOŚĆ O WYBORACH PASUJE DO GRUPY O REZYGNACJI?

W tym fikcyjnym przykładzie system połączył już dwa nagłówki o rezygnacji Macrona w grupę A. Na rysunku oznaczamy je A1 i A2. Przychodzi nagłówek B1 o przedterminowych wyborach we Francji. Dotyczy tego samego kraju, ale innego wydarzenia. Wynik porównania wektorów zdecyduje, czy B1 trafi do grupy A.


GRUPA A · ZAPISANA W BAZIE
Rezygnacja Macrona
A1 + A2 · dwa artykuły
Średnia ich wektorów = środek grupy A
NOWY ARTYKUŁ · B1
Przedterminowe wybory we Francji
Wektor nowego tekstu
Porównanie dwóch wektorów
podobieństwo 0,4010
mniej niż wymagane 0,70
DECYZJA PROGRAMU
B1 rozpoczyna osobną grupę B
Grupa A nadal zawiera A1 i A2

Strzałki pokazują kolejność działań. Położenie ramek nie oznacza odległości między wektorami. Wynik 0,4010 pochodzi z obliczeń na wektorach testowych, tych samych co w sekcji „Próbka”.

1. **ZAMIEŃ NOWY TEKST NA LICZBY** Program pobiera wektor nagłówka B1 z pamięci podręcznej. Jeśli nie ma zapisanego wyniku, zleca obliczenie modelowi. `program`
2. **PORÓWNAJ GO ZE ZNANYMI GRUPAMI** Baza porównuje wektor B1 ze środkami grup, które bierze pod uwagę. W tym przykładzie pozostała jedna taka grupa: A, o rezygnacji Macrona. `baza · pgvector`
3. **SPRAWDŹ, CZY WYNIK WYSTARCZA** Podobieństwo B1 do środka A wynosi **0,4010**. Skala biegnie od −1 do 1; wyższy wynik oznacza większe podobieństwo. Program wymaga co najmniej **0,70**, więc B1 nie dołącza do A. `program`
4. **ZAPISZ ARTYKUŁ W OSOBNEJ GRUPIE** Program zakłada grupę B dla wiadomości o wyborach. Ma ona na razie jeden artykuł, więc jej środkiem jest wektor B1. Artykuł otrzymuje identyfikator nowej grupy. `program + baza`

Gdyby następna redakcja przysłała nagłówek B2 o tych wyborach, jego podobieństwo do grupy B wyniosłoby **0,9942**. Ten wynik przekracza próg 0,70, więc program dołączyłby B2 do B. Grupa miałaby dwa artykuły, a jej środek byłby średnią ich wektorów. Sposób obliczenia tej średniej pokazuje następny rysunek.

> **OD STRONY TECHNICZNEJ**
>
> W bazie grupa tematyczna nazywa się klastrem. Zapytanie przeszukuje tabelę `world_event_clusters`. Bierze pod uwagę tylko grupy tego klienta z doniesieniem z ostatnich 14 dni i wektorami z tego samego modelu. Liczy odległość do każdego pasującego środka, bez indeksu przybliżonego, i zwraca do pięciu najbliższych. Program sprawdza najlepszy wynik. Grupa Z z ostatnim doniesieniem sprzed 16 dni nie spełnia warunku, więc poniższe zapytanie zwraca tylko A.
> Środki są zapisane w kolumnie `centroid` typu `vector(768)`. Operator `<=>` zwraca odległość cosinusową. Program przelicza ją na podobieństwo: `1 − odległość`. W tym przykładzie baza zwraca 0,5990, co daje `1 − 0,5990 = 0,4010`. Próg 0,70 jest zapisany w kodzie programu.

```sql
-- krok 2 dla nagłówka B1, bez rzutowań; wiersze cudzych klientów odcina polityka RLS
SELECT id, centroid <=> :wektor AS distance
FROM world_event_clusters
WHERE embedding_model_id = :model AND last_seen_at >= :dzisiaj_minus_14_dni
ORDER BY centroid <=> :wektor
LIMIT 5;

-- wynik: w oknie mieści się jedna grupa
id grupa A · distance 0.5990
→ podobieństwo 1 − 0,5990 = 0,4010, poniżej progu 0,70 → nowa grupa B
(grupa Z nie spełnia warunku na datę, więc nie ma jej w wyniku)
```

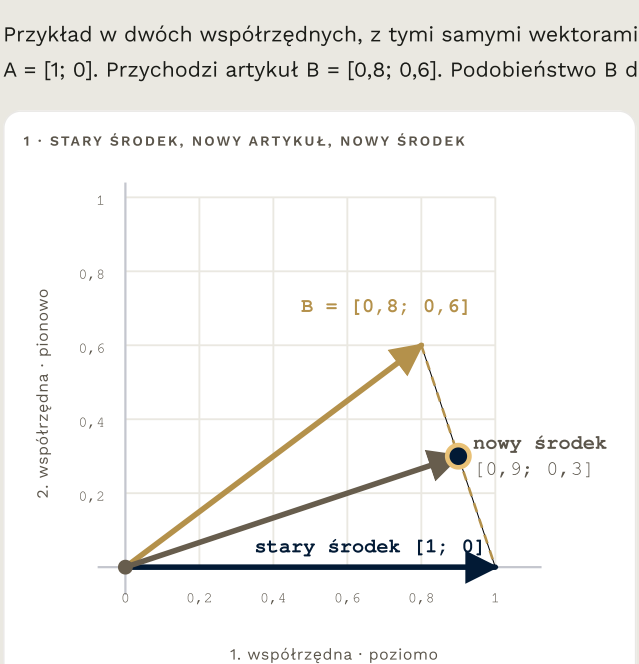
> Zapytanie do działającej bazy z rozszerzeniem pgvector, wykonane na wektorach testowych, zwróciło odległość 0,5990. Obliczenie odległości dla tej samej pary w PHP daje 0,5990. Zgodność obu wyników sprawdza test integracyjny, bo gdyby baza i program liczyły cosinus inaczej, próg 0,70 znaczyłby w każdym z nich co innego.

JAK PRZESUWA SIĘ ŚRODEK

Po dołączeniu artykułu środek przesuwa się w stronę jego wektora. Im więcej artykułów grupa już ma, tym mniejszy wpływ pojedynczego doniesienia. Do obliczenia nowego środka wystarczą poprzedni środek, liczba artykułów i nowy wektor.

> **OD STRONY TECHNICZNEJ**
>
> Przykład w dwóch współrzędnych, z tymi samymi wektorami co w sekcji „Elementarz”. Grupa ma jeden artykuł, więc jej środkiem jest wektor A = [1; 0]. Przychodzi artykuł B = [0,8; 0,6]. Podobieństwo B do środka policzyliśmy już wcześniej: 0,80.

1 · STARY ŚRODEK, NOWY ARTYKUŁ, NOWY ŚRODEK



Nowy środek leży dokładnie w połowie drogi, bo przy jednym artykule obie wagi wynoszą ½. Przy dziesięciu artykułach nowy wektor przesunąłby środek tylko o jedną jedenastą drogi.

2 · POLICZ NOWY ŚRODEK

$$\text{środek}' = \frac{n \times \text{środek} + B}{n + 1}$$

1. Sprawdź próg. Podobieństwo B do środka wynosi 0,80, czyli co najmniej 0,70, więc B dołącza do grupy.
2. Podstaw n = 1, bo grupa miała jeden artykuł: $(1 \times [1; 0] + [0{,}8; 0{,}6]) \div 2 = [1{,}8; 0{,}6] \div 2 = [0{,}9; 0{,}3]$
3. Licznik rośnie do 2, a data ostatniego doniesienia przesuwa się na dziś. Długość nowego środka to $\sqrt{(0{,}81 + 0{,}09)} \approx 0{,}95$, mniej niż 1. Podobieństwo cosinusowe nadal można obliczyć, bo we wzorze dzielimy przez długości wektorów.

[0,9; 0,3] Nowy środek jest średnią obu wektorów. Baza aktualizuje go razem z licznikiem artykułów w jednej instrukcji, więc równoczesne dołączenia artykułów nie nadpisują wzajemnie swoich wyników.

W tabeli grup wektory mają 768 współrzędnych. Instrukcja `UPDATE` aktualizuje je wszystkie naraz.

JAK DZIAŁAJĄ GRUPY TEMATYCZNE

TA SAMA GRUPA PRZEZ WIELE DNI
stały identyfikator
Wtorkowe doniesienie dołącza do grupy założonej w poniedziałek i nie powstaje nowy zbiór. Jeśli grupa idzie do analizy, model widzi oba doniesienia, każde z datą publikacji.

DRUGI PRZEBIEG TEGO SAMEGO DNIA
bez nowych artykułów
Grupa, której nic nie przybyło, nie wraca do analizy.

BRAK NOWYCH DONIESIEŃ PRZEZ 14 DNI
poza oknem
Późniejsze doniesienie rozpoczyna nową grupę. To celowa decyzja: kryzys w marcu i w październiku dają bliskie wektory, ale dla sygnału rynkowego są to dwie sytuacje z innymi cenami wyjściowymi. Okno zamyka grupę tylko dla nowych artykułów. Te, które już do niej trafiły, zostają w niej na stałe.

> **OD STRONY TECHNICZNEJ**
>
> Próg 0,70 wymaga jeszcze sprawdzenia dla dopasowania artykułu do środka grupy. Dwa równoczesne przebiegi jednego klienta mogą założyć tę samą grupę dwa razy i spowodować dodatkową analizę.

PRÓBKA

JAK WYGLĄDA ZAPISANY WEKTOR

Nagłówek o rezygnacji Macrona jest fikcyjny i powstał na potrzeby scenariusza testowego. Wektor zawiera wartości rzeczywiście obliczone przez działający model dla tego nagłówka i zapisane w danych testowych repozytorium. System produkcyjny zapisuje wektory w tym samym formacie. Jeden nagłówek na wejściu, jeden wiersz 768 liczb na wyjściu.

JEDEN NAGŁÓWEK, JEDEN WIERSZ

W tym przypadku pole treści było puste, więc model otrzymał sam nagłówek. Klucz składa się z jego skrótu i nazwy modelu.

CO PRZECZYTAŁ MODEL	Macron resignation abdication power president france Macron resignation
SKRÓT TEKSTU	7319e9beaf703b2351870019ae5446cf94cefe520f5f997f6326b3a357d62169
MODEL	nomic-embed-text-v2-moe
WYNIK	768 liczb · długość 1,00 · 5745 bajtów w wierszu

OD STRONY TECHNICZNEJ

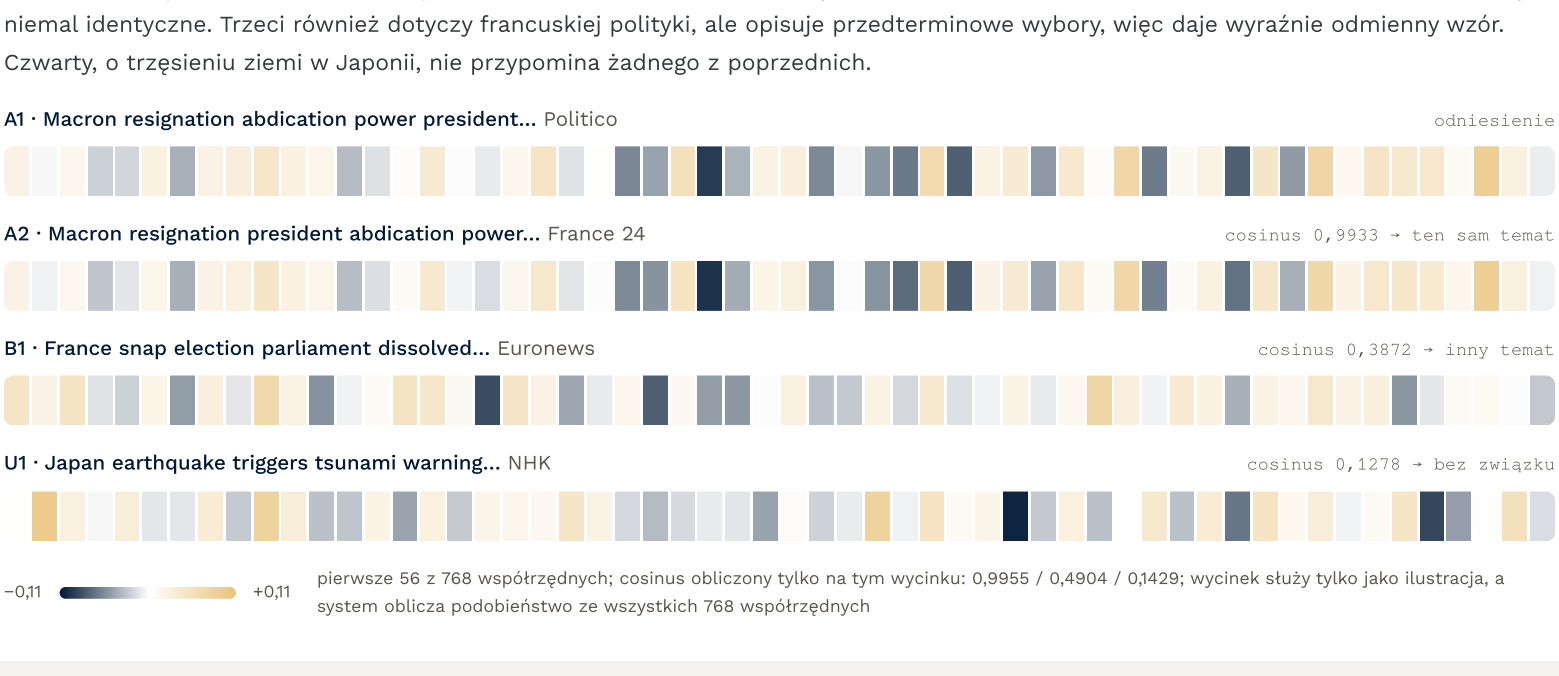
WIERSZ NA DYSKU

Tabela ma trzy kolumny: `VARCHAR(64)`, `VARCHAR(255)` i `JSONB`, a także złożony klucz główny. Innych indeksów nie potrzeba: wyszukiwanie odbywa się po dokładnym kluczu, nigdy po podobieństwie. Po podobieństwie przeszukiwana jest inna tabela, ze środkami grup; opisuje ją sekcja „Grupy tematyczne”.

```
-- SELECT * FROM embedding_cache LIMIT 1;
text_hash 7319e9beaf703b2351870019ae5446cf…
model_id nomic-embed-text-v2-moe
vector [0.021463, −0.003320, 0.013590, −0.022463,
… 762 kolejne wartości …
−0.026217, 0.054305]
```

PIERWSZE 56 WSPÓŁRZĘDNYCH CZTERECH WEKTORÓW

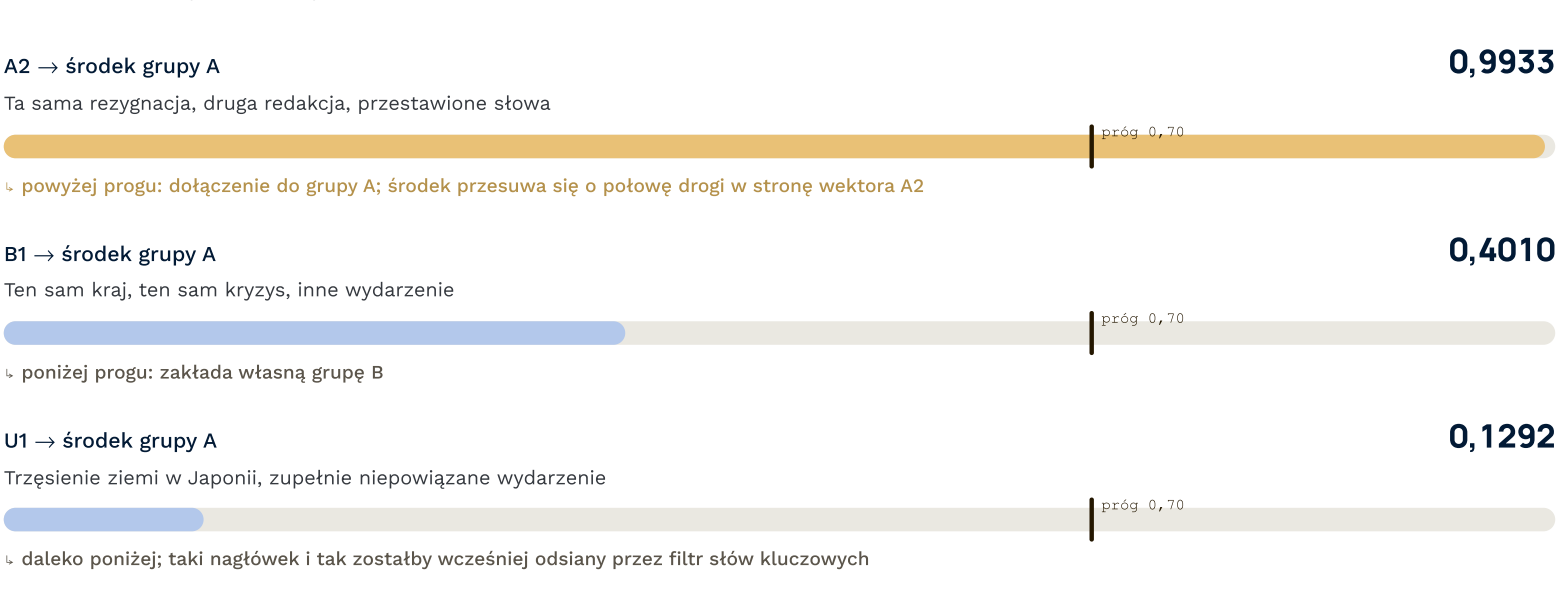
Każdy kwadrat odpowiada jednej współrzędnej. Barwa piaskowa oznacza wartość dodatnią, a granatowa ujemną. Im intensywniejsza barwa, tym większa wartość bezwzględna. Dwa pierwsze nagłówki mówią o rezygnacji Macrona innymi słowami, a mimo to ich wzory są niemal identyczne. Trzeci również dotyczy francuskiej polityki, ale opisuje przedterminowe wybory, więc daje wyraźnie odmienny wzór. Czwarty, o trzęsieniu ziemi w Japonii, nie przypomina żadnego z poprzednich.



pierwsze 56 z 768 współrzędnych; cosinus obliczony tylko na tym wycinku: 0,9955 / 0,4904 / 0,1429; wycinek służy tylko jako ilustracja, a system oblicza podobieństwo ze wszystkich 768 współrzędnych

GDZIE WYPADA PRÓG

Program porównuje trzy kolejne artykuły ze środkiem grupy A. Pierwszy zastaje grupę z jednym artykułem, więc jej środkiem jest wtedy wektor A1. Dwa następne zastają ją już z dwoma. Wartości pochodzą z zapytania do bazy z rozszerzeniem pgvector, uruchomionego na wektorach z danych testowych.



Podobieństwo do środka nie jest tym samym co podobieństwo do pojedynczego artykułu. Paski wyżej pokazują podobieństwo do środka grupy: B1 porównany wprost z A1 daje 0,3872, a ze środkiem grupy A, czyli ze średnią A1 i A2, daje 0,4010. Przy tej grupie różnica nie zmienia wyniku, ale przy grupie z kilkunastoma artykułami środek reprezentuje średnią wektorów całej grupy, a nie pojedynczy artykuł.

WYBORY

CZTERY DECYZJE PROJEKTOWE

01 DWA SKRÓTY ZAMIAST JEDNEGO

Każdy artykuł ma skrót treści służący do rozpoznawania duplikatów. Oba skróty powstają z nagłówka i treści, ale skrót treści nie może bezpiecznie pełnić funkcji klucza pamięci podręcznej. Po zmianie sposobu składania tekstu przekazywanego modelowi skrót służący do wykrywania duplikatów pozostałby bez zmian, choć model dostałby inny tekst. Pamięć podręczna zwracałaby wtedy liczby obliczone dla innej treści, bez żadnego ostrzeżenia. Osobny skrót wymaga zaledwie kilku linii kodu i zapobiega takiej sytuacji.

02 ZATRZYMANIE ZAMIAST PRZEJŚCIA NA INNĄ METODĘ

Gdyby przy niedostępnym modelu program przechodził na porównywanie słów, nadal zwracałby wiarygodnie wyglądające grupy. Byłyby jednak ułożone inaczej, a nic w wyniku nie wskazywałoby, za pomocą której z dwóch metod powstały. Grupy artykułów służą tutaj do tworzenia sygnałów, które mogą później wspierać decyzję transakcyjną podejmowaną poza aplikacją. Błędne pogrupowanie może więc prowadzić do błędnego sygnału. Bezpieczniej zatrzymać program z czytelnym komunikatem, niż przeoczyć pogorszenie jakości wyniku.

03 JEDNA METODA GRUPOWANIA WE WSZYSTKICH TRYBACH

Ustawienie w konfiguracji określa wyłącznie, czy program może wywołać model; nie zmienia metody grupowania. Testy i tryb pracy bez dostępu do sieci korzystają z wyników zapisanych razem z kodem. Przetwarzają je tak samo jak system produkcyjny. Ta sama metoda obejmuje oba rodzaje analizy: podczas obserwacji dłuższego procesu program grupuje artykuły według identyfikatora nadanego w kroku 05. Nie tworzy osobnych klastrów.

> **OD STRONY TECHNICZNEJ**
>
> `OLLAMA_ENABLED` określa, czy przy braku wyniku w pamięci podręcznej wolno wywołać model; nie wybiera strategii grupowania. Dane testowe z pliku `tests/fixtures/embeddings/nomic-embed-text-v2-moe.json` są wczytywane do `embedding_cache` przed każdym scenariuszem. We wszystkich trzech trybach wywoływana jest klasa `CachedTextVectorizer`.

04 DWA MAGAZYNY WEKTORÓW, DWA RÓŻNE PYTANIA

Pamięć podręczna sprawdza po dokładnym kluczu, czy wektor tekstu został już obliczony. Nie wyszukuje po podobieństwie i przechowuje wektory w zwykłym JSON-ie. Indeks podobieństwa wymagałby aktualizacji przy każdym zapisie, choć żadne zapytanie by go nie używało. Stały rozmiar wektora uniemożliwiłby przechowywanie wyników z wielu modeli naraz.

Tabela grup służy do wyszukiwania najbliższej grupy, dlatego zawiera kolumnę typu **vector** z rozszerzenia pgvector.

SZERZEJ

DO CZEGO JESZCZE UŻYWA SIĘ EMBEDDINGÓW

W każdym z wymienionych zastosowań mechanizm jest ten sam. Model umieszcza treści w jednej przestrzeni, a system porównuje ich wektory za pomocą prostej operacji matematycznej.

opisane tutaj
GRUPOWANIE I WYKRYWANIE TEMATÓW
Teksty bez przypisanych kategorii układają się w skupiska, a każde z nich odpowiada jednemu tematowi, także takiemu, którego nikt wcześniej nie przewidział.

opisane tutaj
DOPASOWANIE MIĘDZY JĘZYKAMI
Doniesienia po francusku, angielsku i polsku mogą trafić do jednej grupy bez wcześniejszego tłumaczenia.

powszechne
WYSZUKIWANIE SEMANTYCZNE
Model umieszcza pytanie i dokumenty w tej samej przestrzeni, więc w odpowiedzi na pytanie „jak oddać zepsuty towar” wyszukiwarka podaje „procedurę reklamacyjną”, choć te dwa sformułowania nie mają wspólnego słowa.

powszechne
DOSTARCZANIE MODELOWI KONTEKSTU (RAG)
Zanim pytanie trafi do dużego modelu, system dołącza do niego fragmenty własnych dokumentów, wybrane na podstawie podobieństwa do pytania. Model może wtedy oprzeć odpowiedź na materiale organizacji, zamiast polegać wyłącznie na własnej pamięci.

powszechne
DUPLIKATY, KTÓRE NIE SĄ IDENTYCZNE
Może to być przeklejone ogłoszenie ze zmienionym numerem telefonu albo ta sama usterka zgłoszona po raz drugi innymi słowami. Skrót treści uzna je za różne, ale embedding może pomóc je ze sobą powiązać.

powszechne
KLASYFIKACJA BEZ TRENOWANIA
Każdą kategorię opisuje jedno zdanie, a tekst trafia do tej, której opis leży najbliżej. Dodanie kategorii wymaga kolejnego zdania, a nie ponownego zbierania danych i trenowania klasyfikatora.

powszechne
REKOMENDACJE I „PODOBNE DO TEGO”
System znajduje najbliższych sąsiadów w przestrzeni i na tej podstawie podpowiada podobne artykuły, produkty, oferty pracy czy zgłoszenia. Nie wymaga to ręcznego utrzymywania taksonomii, czyli uporządkowanego systemu kategorii.

powszechne
KIEROWANIE ZGŁOSZEŃ
Nowe zgłoszenie trafia do zespołu, którego dotychczasowe zgłoszenia leżą najbliżej. System opiera się na tej historii, nie na zestawie reguł.

powszechne
WYCHWYTYWANIE ODSTĘPSTW
Tekst odległy od wszystkich dotychczasowych materiałów może być przypadkowym szumem albo pierwszą oznaką czegoś nowego. W obu przypadkach warto mu się przyjrzeć.

✕ CZEGO EMBEDDING NIE ZROBI

Nie ustala, czy dwa teksty są identyczne. Aby sprawdzić, czy ktoś przysłał dokładnie ten sam tekst, wystarczy skrót treści z kroku 01. Sam wynik nie jest też wyjaśnieniem: pokazuje, że dwa teksty są blisko siebie, ale nie mówi, dlaczego. Model korzysta z zależności, których nauczył się podczas trenowania, ale nie sprawdza faktów ani nie szuka innych doniesień. Jeśli artykuł opisuje wydarzenie z zeszłego tygodnia, embedding powstaje wyłącznie z jego treści, bez informacji pobieranych z innych źródeł.

Port `MarketSignals/Domain/WorldEvent/TextVectorizerInterface` **Adaptery** `CachedTextVectorizer → OllamaTextVectorizer`
Pamięć podręczna `embedding_cache (text_hash, model_id) → jsonb` **Grupy tematyczne** `world_event_clusters.centroid → vector(768), operator <=>`
Model `nomic-embed-text-v2-moe · 768 wymiarów` **Inicjalizacja pamięci podręcznej** `bin/console app:embed:warm --days=1`